Databook Client Portal

Management briefing • 29 October 2026 • Architecture proposal / not yet approved

Decision requested: approve a partner-facing MVP using fictional data and a controlled Microsoft test environment; authorize technical discovery with ACP IT before production.

1. Executive summary

ACP consultants prepare Interactive Databooks in Power BI Desktop. The proposed portal lets consultants create a client project, invite permitted viewers, upload a finished .pbix, preview it, and release it. External clients sign in with multifactor authentication and view their authorized Databooks within a branded ACP website; no Power BI authoring tools are required for viewers.

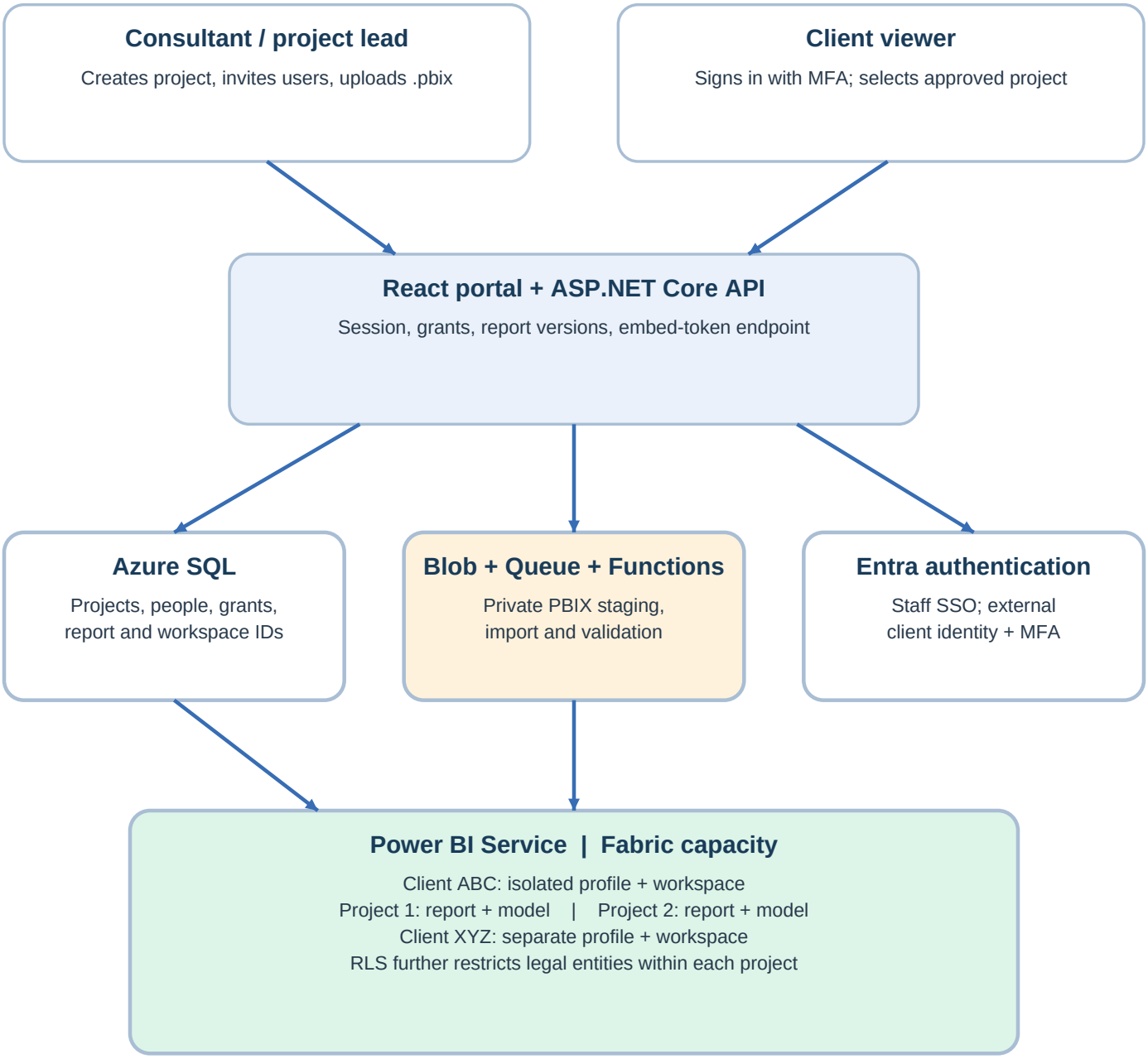
The architecture separates the interactive website/API from background publishing work. Azure Blob Storage provides private, temporary holding for uploaded files. Azure Queue Storage and Azure Functions perform import, validation, release preparation, reminders, and cleanup reliably. Microsoft Power BI Service hosts the actual reports and semantic models. This is the app-owns-data embedding pattern.

2. Who does what

Actor	Normal workflow / responsibility
Consultant / project lead	Creates project; selects or requests client; grants approved people access; uploads .pbix; previews draft; arranges second-person approval and releases.
Client viewer / client administrator	Signs in with MFA; views only released, permitted projects and legal entities; designated client admin may invite colleagues within their own scope.
Sejal / data-engineering & platform owner	Maintains Databook template, company code conventions and Power BI RLS definitions; oversees failed validations, template versions, and platform health. Not a manual publishing bottleneck.
ACP IT / tenant administrators	Own Azure subscription, identity tenants, licensing/capacity, security approvals, production environment, operational permissions.
Portal engineering	Builds user experience, authorization database and API, Power BI provisioning/embedding integration, file processing jobs, audit and tests.

Architecture at a glance

Two paths: publishing a Databook and securely viewing a report



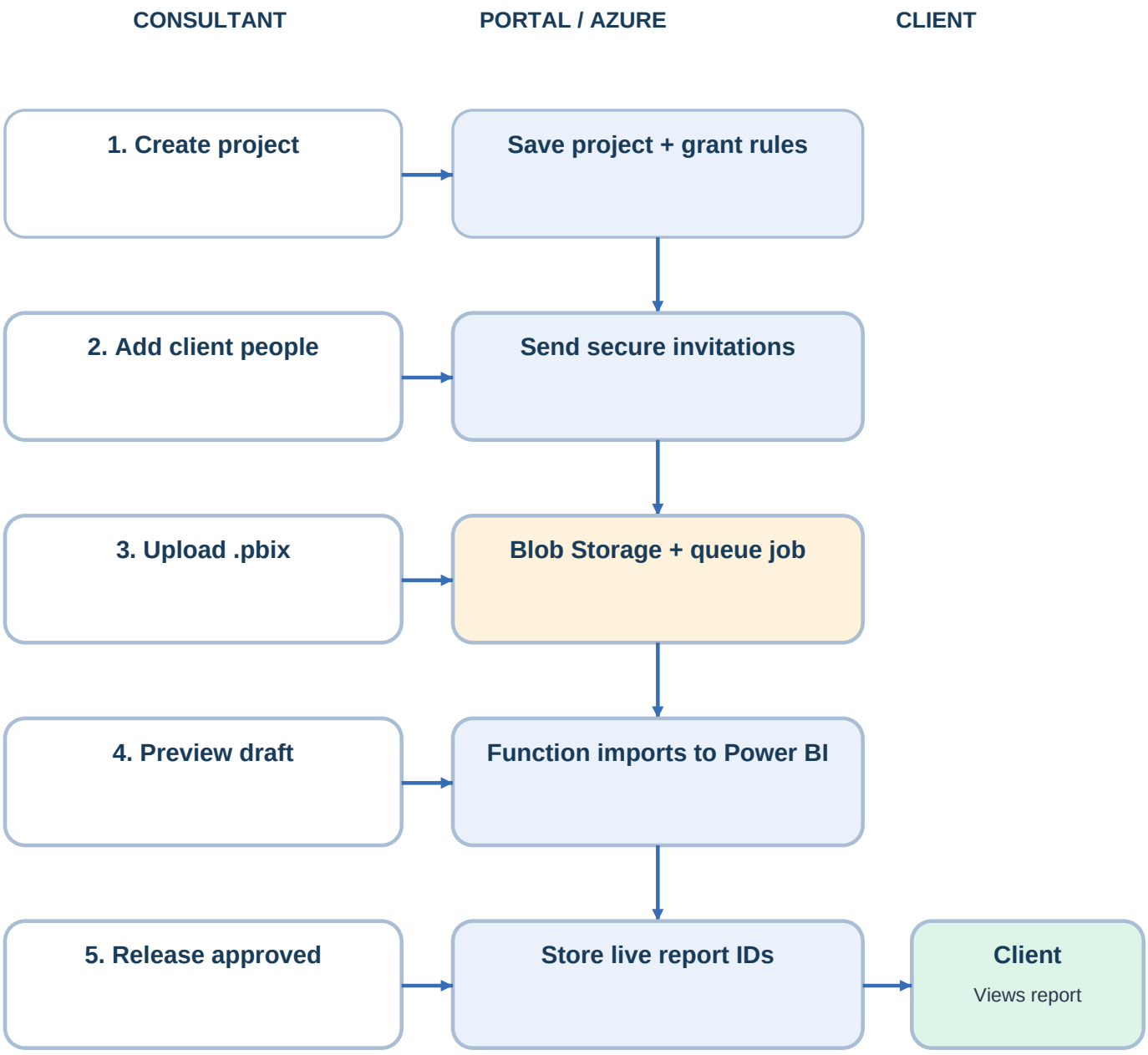
Key principle

The API authorizes access. Power BI hosts and renders the actual report.

Azure Functions handles slow background jobs, not the user's live report request.

Automated publishing & viewing

What the consultant does versus what the platform does automatically



What happens during a PBIX upload?

Blob stores the uploaded file privately; a queue triggers the Azure Function. The Function imports the PBIX into the correct Power BI workspace, checks the template and RLS, then stores report/model IDs for the portal.

Manager note: the consultant never needs to create workspaces or manage Power BI permissions manually.

3. High-level architecture

Layer	Services	What happens
Users / web	React + TypeScript; ASP.NET Core app hosted on Azure App Service	ACP staff and clients access one branded portal; backend authenticates sessions, enforces grants and returns embedded report configuration.
Identity	ACP Microsoft Entra ID; separate Entra External ID tenant for clients	Staff use corporate sign-in; external clients use invitation-led login with MFA. Backend holds session in secure HttpOnly cookie.
Application records	Azure SQL Database	Clients, legal entities/company codes, projects, user memberships, access grants, uploaded versions, live report IDs and audit events.
Upload staging	Private Azure Blob Storage	Browser uploads a .pbix with tightly scoped short-lived upload permission; blob is not public and is retained only as required.
Background processing	Azure Queue Storage + Azure Functions	Queue-triggered import/validation jobs; timer-triggered expiry, reminders and cleanup. Retries, failure tracking and monitoring.
Analytics	Power BI Service on supported Fabric capacity	One workspace and service-principal profile per client; separate report/model per project; embed tokens and RLS enforce permitted views.
Operations	Managed identity, Key Vault, Application Insights, infrastructure as code	Secure API credentials, monitoring and repeatable deployment.

Illustrative trust/data flow: Consultant → Web portal → private Blob upload → queue → Azure Function → Power BI import/validation → draft/release record in SQL. Client → login → backend access check → Power BI embed-token API → interactive report in the browser.

4. Client onboarding and workspace lifecycle

- Authorized user creates a client record (for MVP, administrator-approved to avoid duplicates). The backend provisions a client-scoped service principal profile and workspace in Power BI and stores their IDs in Azure SQL.
- When another project is created for the same client, the backend REUSES that client workspace; its new .pbix becomes a separate semantic model/report pair.
- A client with multiple legal entities can have different viewing scopes. Project grants determine which users can view which project; Power BI RLS enforces which company-code rows are visible inside the project.
- Access expires with project/grant dates. A background timer manages notifications, expiry and retention, subject to ACP-approved policy.

5. Upload, Azure Blob Storage and Azure Functions — detailed flow

Step	System action / output
1. Upload request	Consultant chooses project and .pbix. API checks project role, file type/size and returns a short-lived write-only Blob SAS for one upload.
2. Private staging	Browser uploads directly to private Azure Blob Storage (rather than sending a potentially large PBIX through the web server). API records UploadId and queues processing after completed upload.
3. Queue-triggered Function	Azure Function reads upload metadata, takes a processing lock, downloads/streams the staged file, and calls Power BI Import API as the client-scoped application profile.
4. Import status	Function polls/queries import status, records Power BI report and model IDs, retries transient failures, prevents duplicate imports, and puts hard failures into a review queue.
5. Validate	Function checks allowed template version, company-code list, expected RLS roles, checks/measures and report configuration. Detailed model inspection is proposed through XMLA/TOM; feasibility and required capacity must be proven.
6. Draft and release	Passed version becomes draft; project lead previews with a client identity. A separate authorized approver releases it; only released version is exposed to clients.
7. Post-release	Function can create/update KPI snapshot, send release notices and process scheduled reminders, access expiry, old-upload deletion and retention.

Important: Azure Blob Storage is staging for the .pbix; it is NOT where clients query Power BI visual data. Once imported, Power BI Service hosts the semantic model and report. For very large .pbix files, Microsoft has a separate temporary-upload-location API; evaluate file sizes before choosing the transfer method.

6. Viewing and data isolation

- Client signs in. The application resolves its user ID, active project grant, report version and permitted company codes server-side.
- The backend calls Power BI GenerateToken V2 for the exact report and semantic model, using the profile of that client and an appropriate effective identity/RLS role.
- The React Power BI SDK renders the interactive report in a managed iframe. The browser never receives the app/service principal credential.
- Company identifiers are stable codes in the Databook (the current proposal uses the FIN Tool “Ges.” code). The template needs Client and ACP roles; client RLS filters rows by permitted codes.
- Client workspaces provide tenant isolation; project/report scoping and RLS provide further isolation. Filters or hidden pages alone are NOT security controls.

7. Security and governance

- Explicit project grants (Lead, Member, Viewer, Client) rather than automatic access from seniority. Restricted engagements remain visible only to named members.
- Admin access to platform configuration does not imply silent access to client financials; emergency access must be time-limited and audited.
- Use managed identity in Azure where supported; limit every service principal profile to its own client workspace; do not persist credentials in the browser.
- Blob containers private, short-lived SAS, malware/content checks as applicable, configurable cleanup, encryption in transit/at rest; logs must omit financial values, secrets and embed tokens.
- Every authorization route must be tested across clients and projects; revoking access blocks new embed tokens, while previously issued tokens remain effective until expiry.
- ACP IT and privacy stakeholders must approve EU data residency, retention, security posture, client invitation policy and operational ownership.

8. MVP and success criteria

Partner demonstration scope: two fictional clients, two projects for one client, one consultant/project lead, a second approver and two external viewers; one authentic template-based Databook with 3–4 report pages or report types. The demonstration should exercise the real integration, not a simulated iframe.

MVP feature	Demonstrable outcome
Login and grants	Consultant and client accounts authenticate; cross-client requests are refused.

Client / project creation	First client automatically provisions Power BI profile + workspace; second project reuses it.
PBIX upload & checks	Private Blob upload starts a queue-triggered Function; file imports to Power BI; errors are visible.
Preview / release	Draft is checked and previewed; release requires a distinct approver.
Client reporting	Client A views only its company-code rows; Client B cannot open Client A report.
Operations	Upload status, failed jobs, access log and token expiry are observable.

Not in first MVP: automated migration of real client files, complex client self-administration, advanced exports, extensive portfolio reporting, and production capacity optimization. These should follow security validation and a limited pilot.

9. Dependencies and decisions before build

Needed from	Specific input / approval
Sejal	Sanitized .pbix template and fake-data file; real report/page inventory; company-code field and RLS roles; validation measures and template version conventions.
ACP management	Approve MVP scope, data ownership, information barriers, second-person release and budget envelope.
ACP IT	Test/production tenant access; app identity and profile settings; workspace-creation/API authorization; Azure subscription; EU region; sign-in configuration and deployment approval.
Microsoft licensing / capacity owner	Verify embedding trial behavior, XMLA feature availability and F-SKU fit with largest model; choose production capacity only after a representative load test.

10. Proposed implementation sequence

Gate 1: prove service-principal profile → workspace creation → PBIX import and a working client-scoped RLS embed using fictional data. Gate 2: implement uploads through Blob + queue-triggered Function, validation, draft/release. Gate 3: add styled portal, project/user workflows and admin health. Gate 4: security tests, EU hosting, operational handover and capacity pilot.

Architecture status: proposed. The revised architecture review supersedes earlier suggestions of a single shared service principal, B2B guest client identities in ACP's tenant, and Azure Functions serving the primary interactive API.

11. Microsoft reference documentation

Power BI multitenant embedding and profiles: https://learn.microsoft.com/en-us/power-bi/developer/embedded/embed-multi-tenancy

Import PBIX into Power BI workspace: https://learn.microsoft.com/en-us/rest/api/power-bi/imports/post-import-in-group

Generate Power BI embed tokens: https://learn.microsoft.com/en-us/power-bi/developer/embedded/generate-embed-token

Embedded RLS: https://learn.microsoft.com/en-us/power-bi/developer/embedded/cloud-rls

Azure Functions and Storage triggers: https://learn.microsoft.com/en-us/azure/azure-functions/storage-considerations

Azure Queue trigger: https://learn.microsoft.com/en-us/azure/azure-functions/functions-bindings-storage-queue

Source basis: ACP Databook Client Portal Product Brief (26 Oct 2026) and Databook Portal Architecture Review (28 Oct 2026), supplied for this proposal. Microsoft documentation links support the technical integration choices; actual tenant configuration and costs remain to be validated.